

UNIVERSIDAD DE SANTIAGO DE CHILE
FACULTAD DE INGENIERÍA
Departamento de Ingeniería Informática



Proyecto de investigación — Informe 2

Maximum Clique: Resolución mediante Metaheurística y Contraste con
Método Exacto

Integrantes: Ricardo Riveros

Gonzalo Ahumada

Daniel Muñoz

Asignatura: Optimización en Ingeniería

Profesores: Mónica Villanueva

Mario Inostroza

Ayudante: Sebastián Aliaga

20 de julio de 2026

Tabla de contenidos

Resumen	1
1. Introducción	2
2. Estado del Arte	5
2.1. Búsqueda local y metaheurísticas para el MCP	5
2.2. Diversificación mediante reinicio: Búsqueda Local Iterada	6
2.3. Contextos de aplicación	7
3. Método	8
3.1. Representación, función objetivo y vecindad (Tarea 1)	8
3.2. Búsqueda Tabú con reinicio por perturbación	9
3.3. Procedimiento experimental	10
3.4. Dificultades experimentales	11
4. Resultados y Análisis	13
4.1. Descripción de las instancias	13
4.2. Resultados agregados de las 30 corridas	13
4.3. Efecto del mecanismo de reinicio (experimento de ablación)	14
4.4. Curvas de convergencia	14
4.5. Visualización de las soluciones	16
5. Contraste de Resultados: Método Exacto vs. Metaheurística	18
5.1. Fuentes de comparación	18
5.2. Tabla comparativa	19
5.3. Análisis del contraste	20
6. Discusión	22
6.1. Efectividad de la Búsqueda Tabú con reinicio	22
6.2. Análisis de las corridas que no alcanzaron el óptimo	22
6.3. Rol del mecanismo de reinicio	23

6.4. Ventajas del enfoque implementado	23
6.5. Comparación con otros diseños de metaheurística considerados	24
6.6. Limitaciones	24
7. Conclusiones	26
7.1. Trabajo futuro	27
8. Apéndice	29
8.1. Entorno de ejecución	29
8.2. Cálculo de la vecindad y construcción voraz	29
8.3. Búsqueda Tabú con reinicio (Algoritmo 1)	30
8.4. Instancias: formato DIMACS	34
Referencias bibliográficas	35

Resumen

Este trabajo da continuidad al estudio del Problema del Clique Máximo (MCP) iniciado en el Informe 1, donde se resolvió de forma exacta mediante un modelo de Programación Lineal Entera (ILP) resuelto por Ramificación y Acotamiento. En esta segunda etapa se aborda el mismo problema mediante una metaheurística de trayectoria: Búsqueda Tabú con reinicio por perturbación (estilo Búsqueda Local Iterada), construida sobre la representación de soluciones y el operador de vecindad Add/Drop/Swap especificados en la Tarea 1 de la asignatura. El algoritmo se implementa en Python y se evalúa mediante 30 corridas independientes sobre dos instancias del benchmark DIMACS: *keller4* (171 vértices, $\omega = 11$), la misma utilizada en el Informe 1, y *C125.9* (125 vértices, densidad 0.90, $\omega = 34$), una instancia más densa sobre la cual un Branch & Bound exacto no logró probar optimalidad dentro de un límite de 300 segundos. Los resultados muestran que la metaheurística alcanza el óptimo conocido en el 93.3 % de las corridas en *keller4* y en el 96.7 % en *C125.9*, con un tiempo medio hasta la mejor solución de apenas 0.021–0.025 segundos, varios órdenes de magnitud menor que el tiempo requerido por las referencias exactas consideradas. El contraste directo entre ambas soluciones implementadas evidencia el compromiso fundamental entre garantía de optimalidad y escalabilidad: el método exacto certifica el óptimo cuando logra cerrar el árbol de búsqueda, mientras que la metaheurística entrega soluciones de calidad prácticamente óptima en tiempos casi constantes, sin certificado de optimalidad. Se concluye que, para instancias donde el enfoque exacto se vuelve computacionalmente inviable, la Búsqueda Tabú con reinicio constituye una alternativa efectiva y de bajo costo computacional.

Palabras clave: Clique Máximo, Búsqueda Tabú, Búsqueda Local Iterada, Metaheurísticas de Trayectoria, Optimización Combinatoria, Benchmark DIMACS, Contraste de métodos.

1. Introducción

El Problema del Clique Máximo (*Maximum Clique Problem*, MCP) consiste en encontrar, dado un grafo no dirigido $G = (V, E)$, el subgrafo completo (clique) de mayor cardinalidad, denotada $\omega(G)$ (Pardalos & Xue, 1994). El problema es NP-duro y, más aún, no admite un algoritmo de aproximación con factor constante en tiempo polinomial salvo que $P = NP$ (Bomze et al., 1999; Pardalos & Xue, 1994), lo que motiva el estudio de estrategias de resolución alternativas a la enumeración exacta cuando el tamaño o la densidad de la instancia lo impiden.

En el Informe 1 se abordó el MCP mediante un modelo de Programación Lineal Entera (ILP), recordado aquí brevemente por autocontención: con $x_i \in \{0, 1\}$ indicando si el vértice i pertenece al clique, el modelo es $\max \sum_{i \in V} x_i$ sujeto a $x_i + x_j \leq 1 \ \forall (i, j) \notin E$ (formulación completa y su relajación LP en Informe 1, Sección 3.1). Este modelo fue resuelto de forma exacta con el solver HiGHS (Huangfu & Hall, 2018) sobre la instancia *keller4* (171 vértices, 9 435 aristas), obteniendo el óptimo $\omega = 11$ en 12–18 segundos. Ese mismo trabajo evidenció, sin embargo, la fragilidad de este enfoque frente a la dificultad de la instancia: la relajación lineal (*Linear Programming*, LP) presentó una brecha de integralidad del 87.1 %, y un solver menos sofisticado (GLPK, *GNU Linear Programming Kit*) no logró probar optimalidad dentro de 300 segundos sobre el mismo modelo. Exploraciones adicionales con un Branch & Bound implementado manualmente sobre una instancia más densa, *C125.9* (125 vértices, densidad 0.90, $\omega = 34$), confirmaron esta limitación: tras explorar 232 953 nodos del árbol de búsqueda durante 300 segundos, el algoritmo encontró el clique óptimo pero *no logró certificar su optimalidad*. Estos antecedentes sitúan el punto de partida de este segundo informe: ¿qué ocurre cuando se abandona la garantía de optimalidad a cambio de un método capaz de escalar a instancias donde el enfoque exacto se degrada?

En la Tarea 1 de la asignatura se diseñaron los componentes fundamentales de una metaheurística de búsqueda local para el MCP: una representación de soluciones como subconjuntos explícitos de vértices que son, por construcción, siempre cliques válidos; una función objetivo trivial $f(S) = |S|$; y un operador de vecindad compuesto por

tres movimientos —ADD, DROP y SWAP— que preserva la factibilidad en cada paso. Este informe construye directamente sobre ese diseño, instanciándolo en una metaheurística concreta: **Búsqueda Tabú con reinicio por perturbación** (de inspiración en Búsqueda Local Iterada, ILS), y la evalúa computacionalmente para contrastar sus resultados con los del método exacto del Informe 1.

Objetivo general. Diseñar, implementar y evaluar una metaheurística de búsqueda local para el MCP sobre la vecindad Add/Drop/Swap de la Tarea 1, y contrastar sus resultados —calidad de solución y tiempo de cómputo— con los del método exacto (ILP + Branch & Bound) del Informe 1.

Objetivos específicos.

- Implementar en Python una Búsqueda Tabú con lista tabú, criterio de aspiración y reinicio por perturbación, sobre la representación y vecindad especificadas en la Tarea 1.
- Evaluar el algoritmo mediante corridas independientes sobre *keller4* (la misma instancia del Informe 1) y sobre *C125.9*, una instancia más densa donde el método exacto no logró certificar optimalidad.
- Cuantificar la tasa de éxito, el tiempo hasta la mejor solución y la calidad relativa de las soluciones obtenidas respecto al óptimo conocido de cada instancia.
- Contrastar sistemáticamente los resultados del enfoque exacto (Informe 1) y del enfoque metaheurístico (este informe), identificando en qué condiciones cada uno resulta preferible.

Pregunta de investigación. ¿En qué medida una metaheurística de búsqueda local relativamente simple, construida sobre una vecindad de tres movimientos que preserva la factibilidad por construcción, logra igualar la calidad de un método exacto para el MCP mientras reduce drásticamente el tiempo de cómputo, y qué se sacrifica —en términos de garantías— al hacerlo?

El resto del documento se organiza como sigue: la Sección 2 presenta el estado del arte de las metaheurísticas para el MCP; la Sección 3 describe el método, retomando el diseño de la Tarea 1 y especificando el algoritmo Tabú+ILS; la Sección 4 presenta los resultados computacionales; la Sección 5 contrasta explícitamente ambos informes; y finalmente se presentan la discusión y las conclusiones.

2. Estado del Arte

El Informe 1 revisó la literatura sobre complejidad y métodos exactos del MCP (Branch & Bound, formulaciones ILP y su relajación de Motzkin-Straus). Esta sección se enfoca en los enfoques metaheurísticos, que constituyen el ámbito de este segundo informe.

2.1. Búsqueda local y metaheurísticas para el MCP

La dificultad práctica de los métodos exactos en instancias grandes o densas —evidenciada en el Informe 1 mediante la brecha de integralidad del 87.1 % en *keller4* y la incapacidad de GLPK de cerrar el árbol de búsqueda en 300 s— motivó desde temprano el desarrollo de heurísticas de búsqueda local para el MCP. Las primeras propuestas son heurísticas constructivas voraces (*greedy*), del tipo *best-in/worst-out*, que construyen o podan un clique vértice a vértice (Pardalos & Xue, 1994). Estas heurísticas son rápidas pero quedan atrapadas en óptimos locales tan pronto como el clique construido se vuelve maximal, lo que ha motivado dos líneas de mejora principales:

1. **Búsqueda Tabú (Tabu Search).** Introducida por Glover (1989, 1990) y sistematizada en Glover y Laguna (1997), la Búsqueda Tabú evita ciclar entre óptimos locales prohibiendo temporalmente revertir movimientos recientes mediante una *lista tabú*, permitiendo excepciones cuando un criterio de *aspiración* se satisface (por ejemplo, si el movimiento conduce a una solución mejor que la mejor conocida). Battiti y Protasi (2001) aplicaron esta idea directamente al MCP mediante *Reactive Local Search*: alternan entre agregar vértices (mientras el clique no sea maximal) y realizar movimientos de DROP/SWAP controlados por una lista tabú cuya duración (*tenure*) se adapta dinámicamente según la frecuencia de repetición de soluciones —una extensión conocida como *Reactive Tabu Search* (Battiti & Tecchiolli, 1994)—, reportando resultados competitivos en el benchmark DIMACS con tiempos de cómputo muy inferiores a los métodos exactos.

2. **Búsqueda local dinámica y basada en penalización.** Pullan y Hoos (2006)

proponen *Dynamic Local Search* (DLS-MC), que penaliza dinámicamente los vértices no seleccionados para diversificar la búsqueda sin necesidad de una lista tabú explícita, alternando fases de expansión (ADD) y de *plateau search* (movimientos que no cambian $f(S)$ pero sí el conjunto de candidatos, análogos al operador SWAP de la Tarea 1). En una línea similar, Katayama et al. (2005) combinan una búsqueda de vecindad variable con una estrategia de diversificación basada en el historial de vértices visitados, mejorando la robustez frente a instancias con muchos óptimos locales de calidad similar. Singh y Govinda (2014) proponen heurísticas más simples basadas en el grado de los vértices dentro de conjuntos independientes mínimos, priorizando velocidad de cómputo sobre robustez.

Un elemento común a estos trabajos —y que se retoma directamente en este informe— es que todos operan sobre una representación de solución que es, por construcción, siempre un clique válido (a diferencia de representaciones binarias penalizadas, más habituales en algoritmos genéticos), evitando así el costo de reparar soluciones infeasibles en cada iteración. Esto coincide con el diseño de representación y vecindad Add/Drop/Swap especificado en la Tarea 1 (Sección 3).

2.2. Diversificación mediante reinicio: Búsqueda Local Iterada

Un riesgo compartido por las búsquedas locales de trayectoria única (incluida la Búsqueda Tabú) es el estancamiento prolongado en una región del espacio de búsqueda cuando la lista tabú por sí sola no basta para escapar de un conjunto de óptimos locales mutuamente cercanos. El marco de *Búsqueda Local Iterada* (*Iterated Local Search*, ILS) de Lourenço et al. (2003) aborda este problema intercalando, tras cada fase de búsqueda local, una *perturbación* de la solución incumbente (típicamente la eliminación aleatoria de una fracción de la solución) seguida de una reconstrucción, lo que permite explorar una nueva región del espacio sin perder por completo el progreso acumulado. Este informe combina ambas ideas —lista tabú para la intensificación local y perturbación tipo ILS para la diversificación global— en un único algoritmo, descrito en la Sección 3.

2.3. Contextos de aplicación

Los dos contextos de aplicación del MCP identificados en el Informe 1 —diseño de códigos correctores de errores mediante la conjetura de Keller (Lagarias & Shor, 1992), directamente vinculada a la familia de instancias *keller* utilizada en este trabajo, y la búsqueda de subestructuras moleculares comunes en bioinformática (Gardiner et al., 1997)— cobran especial relevancia en el contexto metaheurístico: ambos dominios generan instancias reales cuyo tamaño (n en el orden de cientos a miles de vértices, grafos de producto molecular densos) hace inviable la resolución exacta en tiempo práctico, siendo precisamente el nicho donde las metaheurísticas de búsqueda local como la evaluada en este informe resultan indispensables.

3. Método

3.1. Representación, función objetivo y vecindad (Tarea 1)

Este informe reutiliza íntegramente los componentes de diseño especificados en la Tarea 1 de la asignatura. Se resumen aquí para hacer autocontenido este documento; el detalle completo, con ejemplos numéricos sobre una instancia ilustrativa, se encuentra en dicho trabajo.

Representación. Una solución es un subconjunto explícito de vértices $S \subseteq V$ que constituye, por construcción, un clique de G . El espacio de búsqueda se restringe desde el inicio a $\Omega = \{S \subseteq V : S \text{ es un clique de } G\}$, de modo que toda solución visitada es factible y no se requieren penalizaciones ni reparación.

Función objetivo. $f(S) = |S|$, a maximizar, evaluable en $O(1)$ manteniendo un contador junto con S .

Conjunto de candidatos. Para $S \in \Omega$, $C(S) = (\bigcap_{u \in S} N(u)) \setminus S$: los vértices fuera de S adyacentes a *todos* los vértices de S , que pueden agregarse sin romper la propiedad de clique.

Vecindad $N(S)$. Definida como la unión de tres movimientos, todos cerrados en Ω :

- ADD(v), $v \in C(S)$: $S' = S \cup \{v\}$ (incrementa f en 1).
- DROP(u), $u \in S$: $S' = S \setminus \{u\}$ (decrementa f en 1).
- SWAP(u, v), con $v \notin S$ tal que $\deg_S(v) = |S| - 1$ y $\{u\} = S \setminus N(v)$: $S' = (S \setminus \{u\}) \cup \{v\}$ (mantiene f constante, cambia $C(S)$).

El cálculo de $N(S)$ mediante una matriz de adyacencia A tiene complejidad $O(nk)$, con $n = |V|$, $k = |S|$ (Tarea 1, Algoritmo 1).

3.2. Búsqueda Tabú con reinicio por perturbación

Sobre esta misma vecindad se construye una metaheurística de trayectoria que agrega una regla de selección y aceptación de movimientos: **Búsqueda Tabú** (Glover, 1989, 1990) para la intensificación local, combinada con un mecanismo de **perturbación y reconstrucción** de inspiración en Búsqueda Local Iterada (Lourenço et al., 2003) para la diversificación cuando la búsqueda se estanca.

Idea general. En cada iteración se prioriza, en orden: (i) si $C(S) \neq \emptyset$, aplicar ADD de un vértice candidato —movimiento siempre mejorante, nunca se rechaza salvo por la lista tabú—; (ii) si S es maximal ($C(S) = \emptyset$) pero existen movimientos SWAP disponibles, aplicar el que maximiza $|C(S')|$ tras el intercambio, favoreciendo estados desde los cuales sea más probable reanudar la expansión; (iii) si no hay ADD ni SWAP disponibles, forzar un DROP aleatorio para diversificar. Un vértice recién eliminado de S (por DROP o como parte de un SWAP) queda *tabú* para volver a agregarse durante **tenure** iteraciones, evitando ciclos cortos de ida y vuelta. Cuando el mejor incumbente no mejora durante **stall_limit** iteraciones consecutivas, se aplica una *perturbación*: se elimina una fracción aleatoria de S (entre 10 % y 30 %) y se reconstruye de forma voraz, análogo al paso de perturbación de ILS.

Manejo de la lista tabú (fallback en lugar de aspiración clásica). Si todos los candidatos de un tipo de movimiento están vetados por la lista tabú, la restricción se levanta temporalmente sobre ese conjunto de candidatos en lugar de bloquear la búsqueda. Para el movimiento ADD esto coincide de hecho con un criterio de aspiración clásico, ya que todo ADD mejora estrictamente $f(S)$; para SWAP y DROP es una salvaguarda práctica que evita el estancamiento total cuando la lista tabú cubre, por azar, la totalidad de los movimientos disponibles.

Este procedimiento se resume formalmente en el Algoritmo 1, presentado a continuación.

Algorithm 1 Búsqueda Tabú con reinicio por perturbación para el MCP

Require: Grafo $G = (V, E)$ (matriz A), `tenure`, `stall_limit`, `max_iter`

Ensure: Mejor clique S^* encontrado

```
1:  $S \leftarrow \text{CONSTRUCCIONVORAZ}(\emptyset)$   $\triangleright$  Add aleatorio hasta  $C(S) = \emptyset$ 
2:  $S^* \leftarrow S$ ;  $\text{tabu\_hasta}[v] \leftarrow 0 \forall v \in V$ ;  $\text{estancado} \leftarrow 0$ 
3: for  $it = 1$  hasta max_iter do
4:   Calcular  $C(S)$  y pares SWAP disponibles  $\triangleright$  Algoritmo 1, Tarea 1,  $O(nk)$ 
5:   if  $C(S) \neq \emptyset$  then
6:      $v \leftarrow$  elegir de  $C(S)$ , prefiriendo no-tabú  $\triangleright$  Add: aspiración natural
7:      $S \leftarrow S \cup \{v\}$ ;  $\text{estancado} \leftarrow 0$ 
8:   else if hay pares SWAP then
9:      $(u, v) \leftarrow \arg \max_{(u,v)} |C(S \setminus \{u\} \cup \{v\})|$  entre no-tabú (o todos, si vacío)
10:     $S \leftarrow (S \setminus \{u\}) \cup \{v\}$ ;  $\text{tabu\_hasta}[u] \leftarrow it + \text{tenure}$ ;  $\text{estancado} += 1$ 
11:  else
12:     $u \leftarrow$  vértice aleatorio de  $S$   $\triangleright$  Drop forzado
13:     $S \leftarrow S \setminus \{u\}$ ;  $\text{tabu\_hasta}[u] \leftarrow it + \text{tenure}$ ;  $\text{estancado} += 1$ 
14:  end if
15:  if  $|S| > |S^*|$  then
16:     $S^* \leftarrow S$ ;  $\text{estancado} \leftarrow 0$ 
17:  end if
18:  if  $\text{estancado} \geq \text{stall\_limit}$  then  $\triangleright$  Perturbación tipo ILS
19:    Eliminar fracción aleatoria (10–30 %) de  $S$ , marcar tabú
20:     $S \leftarrow \text{CONSTRUCCIONVORAZ}(S)$ ;  $\text{estancado} \leftarrow 0$ 
21:  end if
22: end for
23: return  $S^*$ 
```

3.3. Procedimiento experimental

1. **Herramientas de desarrollo.** Se implementó el algoritmo en Python 3.9 con NumPy (Harris et al., 2020) para las operaciones vectorizadas sobre la matriz

de adyacencia, NetworkX (Hagberg et al., 2008) para la visualización de grafos y Matplotlib (Hunter, 2007) para las figuras. A diferencia del Informe 1, no se requieren Pyomo ni un solver MIP (*Mixed Integer Programming*, programación entera mixta), ya que la metaheurística no formula un programa matemático: opera directamente sobre la representación combinatoria de la Sección 3.1.

2. **Instancias de prueba.** Se emplean dos instancias del benchmark DIMACS (Johnson & Trick, 1993; Mascia, 2014): *keller4* (171 vértices, 9 435 aristas, $\omega = 11$), la misma del Informe 1, para comparación directa; y *C125.9* (125 vértices, 6 963 aristas, densidad 0.90, $\omega = 34$), una instancia más densa sobre la cual un Branch & Bound manual (PyBnB) no logró certificar optimalidad en 300 s (Sección 5).
3. **Parámetros.** Tras pruebas preliminares, se fijó `tenure`= 10, `stall_limit`= 200, fracción de perturbación $\in [0.1, 0.3]$ y `max_iter`= 8 000. Estos valores balancean intensificación (`tenure` pequeño permite reincorporar vértices rápidamente) y diversificación (`stall_limit` moderado evita perturbar prematuramente una búsqueda aún productiva).
4. **Protocolo de evaluación.** Para cada instancia se realizaron 30 corridas independientes (semillas 0–29), registrando en cada una: tamaño del mejor clique encontrado, iteración y tiempo en que se alcanzó dicho tamaño, número de perturbaciones aplicadas y tiempo total de ejecución. Cada solución se validó verificando que el subgrafo inducido contiene exactamente $\binom{|S^*|}{2}$ aristas.

3.4. Dificultades experimentales

1. **Riesgo de estancamiento en instancias densas.** En *C125.9* (densidad 0.90), la vecindad SWAP es mucho más numerosa que en *keller4*, lo que en pruebas preliminares (sin mecanismo de perturbación) provocaba que la búsqueda oscilara indefinidamente en un conjunto de cliques maximales de tamaño 32–33 sin alcanzar el óptimo $\omega = 34$. Esto se resolvió incorporando el mecanismo de perturbación

tipo ILS de la Sección 3.2: en una comparación controlada sobre las mismas 30 semillas, desactivar el reinicio (`stall_limit` $\rightarrow \infty$) deja la tasa de éxito en 66.7 % y el tamaño medio del clique en 33.4, mientras que con reinicio la tasa de éxito sube a 96.7 % (Sección 4).

2. **Costo de evaluar SWAP exhaustivamente.** Evaluar $|C(S')|$ para *todos* los pares SWAP candidatos en cada iteración resulta costoso en instancias densas, donde puede haber decenas de pares disponibles. Se optó por muestrear hasta 15 pares por iteración en lugar de evaluarlos todos, sin degradar la calidad de las soluciones finales pero reduciendo notablemente el tiempo por iteración.
3. **Elección de tenure.** Un `tenure` demasiado alto (> 30) bloqueaba una fracción excesiva de vértices tras cada perturbación, degradando la calidad de la reconstrucción voraz posterior; un `tenure` demasiado bajo (< 5) permitía ciclos cortos. El valor `tenure` = 10 se fijó empíricamente tras comparar la tasa de éxito en un barrido preliminar sobre *keller4*.

4. Resultados y Análisis

Se presentan los resultados de 30 corridas independientes de la Búsqueda Tabú con reinicio (Algoritmo 1) sobre *keller4* y *C125.9*, con los parámetros especificados en la Sección 3.

4.1. Descripción de las instancias

La Tabla 1 resume las instancias utilizadas. *C125.9* se incorpora respecto al Informe 1 precisamente porque, siendo de menor tamaño que *keller4*, es considerablemente más densa, lo que la vuelve más difícil para el método exacto (Sección 5) y permite evaluar la metaheurística en un régimen distinto.

Tabla 1: Características de las instancias evaluadas.

Instancia	$ V $	$ E $	Densidad	$\omega(G)$ conocido
keller4	171	9 435	0.6491	11
C125.9	125	6 963	0.8985	34

4.2. Resultados agregados de las 30 corridas

La Tabla 2 resume, para cada instancia, la tasa de éxito (fracción de corridas que alcanzó $\omega(G)$), el tamaño medio del clique encontrado, y el tiempo e iteraciones hasta alcanzar la mejor solución de cada corrida.

Tabla 2: Resultados agregados de la Búsqueda Tabú con reinicio (30 corridas, semillas 0–29).

Métrica	keller4	C125.9
Tasa de éxito ($f(S^*) = \omega(G)$)	93.3 % (28/30)	96.7 % (29/30)
Tamaño medio $\pm$ desv. estándar	10.90 ± 0.40	33.97 ± 0.18
Mejor tamaño observado	11	34
Tiempo medio hasta mejor solución	0.0211 ± 0.0458 s	0.0251 ± 0.0385 s
Iteraciones medias hasta mejor solución	556.3	632.3

En ambas instancias, la metaheurística alcanza el óptimo conocido en la gran mayoría de las corridas. En *C125.9*, la única corrida que no lo alcanza se queda a un solo vértice de distancia ($f(S^*) = 33$). En *keller4*, de las dos corridas que no alcanzan el óptimo, una se queda a un vértice de distancia ($f(S^*) = 10$) y la otra a dos vértices ($f(S^*) = 9$). El tiempo hasta la mejor solución es del orden de centésimas de segundo en ambos casos, con una dispersión (desviación estándar) comparable a la media, lo que refleja que algunas corridas requieren varias perturbaciones antes de alcanzar el óptimo, mientras que otras lo hacen en la primera fase de expansión voraz.

4.3. Efecto del mecanismo de reinicio (experimento de ablación)

Para cuantificar el aporte del mecanismo de perturbación tipo ILS, se repitió el experimento desactivando el reinicio ($\text{stall_limit} \rightarrow \infty$), es decir, ejecutando una Búsqueda Tabú pura sobre las mismas 30 semillas. La Tabla 3 muestra el resultado.

Tabla 3: Experimento de ablación: Búsqueda Tabú pura (sin reinicio) vs. con reinicio tipo ILS.

Variante	keller4		C125.9	
	Tasa éxito	Tamaño medio	Tasa éxito	Tamaño medio
Sin reinicio (Tabú puro)	73.3 %	10.57 ± 0.76	66.7 %	33.40 ± 1.11
Con reinicio (Tabú+ILS)	93.3 %	10.90 ± 0.40	96.7 %	33.97 ± 0.18

El reinicio por perturbación mejora la tasa de éxito en 20 puntos porcentuales en *keller4* y 30 puntos en *C125.9*, y reduce la desviación estándar del tamaño final en un 48 % en *keller4* y en un 84 % en *C125.9*. El efecto es más pronunciado en *C125.9*, consistente con lo señalado en la Sección 3: al ser una instancia más densa, la vecindad SWAP genera más plateaus donde el Tabú puro puede quedar atrapado sin un mecanismo de diversificación explícito.

4.4. Curvas de convergencia

Las Figuras 1 y 2 muestran la evolución del mejor $f(S)$ encontrado a lo largo de una corrida representativa (la que tardó más iteraciones en alcanzar su mejor

solución, para ilustrar el mecanismo de reinicio con la trayectoria más larga y visible) para cada instancia. En ambos casos se observa un patrón característico de *plateaus*: la construcción voraz inicial alcanza rápidamente un clique maximal subóptimo, seguido de largos períodos sin mejora donde la Búsqueda Tabú explora la vecindad SWAP, hasta que una perturbación desplaza la búsqueda hacia una región desde la cual es posible seguir expandiendo.

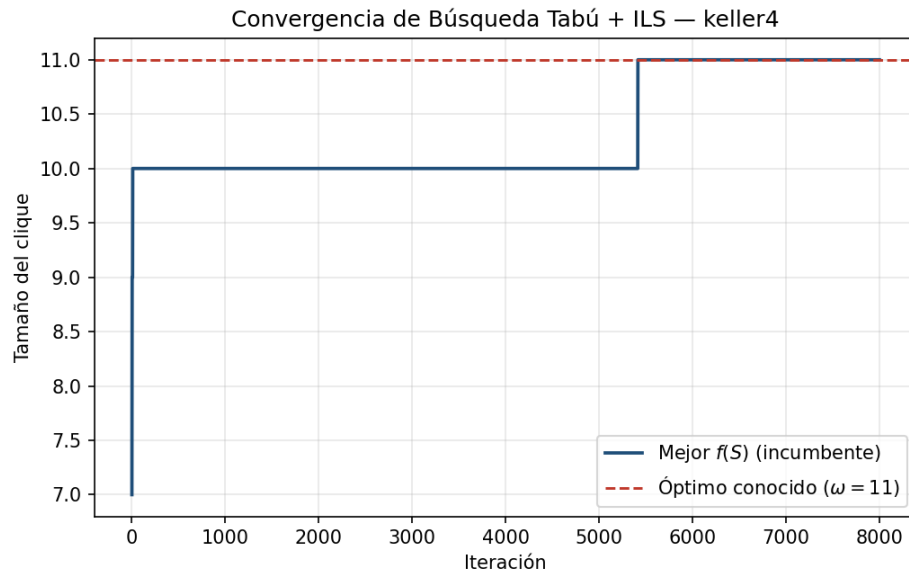


Figura 1: Convergencia de la Búsqueda Tabú+ILS sobre *keller4* (corrida representativa, semilla 20, 39 perturbaciones).

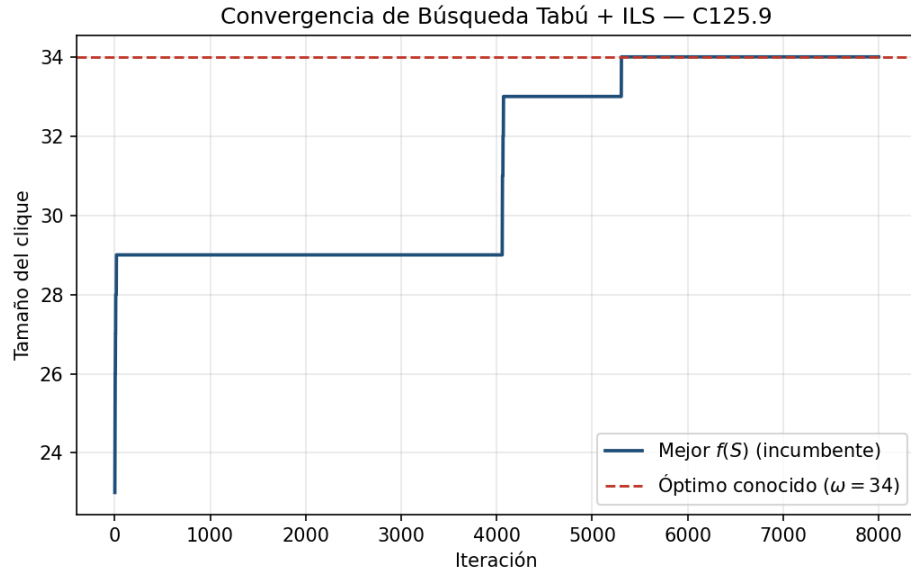


Figura 2: Convergencia de la Búsqueda Tabú+ILS sobre $C125.9$ (corrida representativa, semilla 8, 39 perturbaciones).

4.5. Visualización de las soluciones

Las Figuras 3 y 4 muestran, para cada instancia, el grafo completo con el clique óptimo encontrado resaltado en rojo.

Clique encontrado por Tabú+ILS — *keller4* ($|S|=11$)

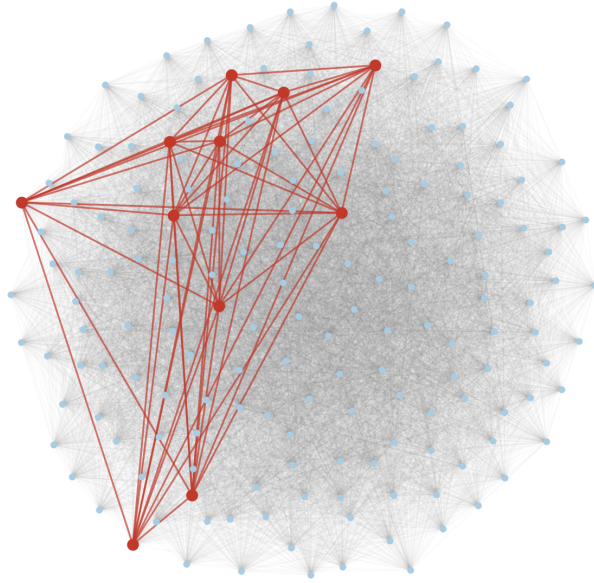


Figura 3: Clique de tamaño 11 encontrado por Tabú+ILS en *keller4*.

Clique encontrado por Tabú+ILS — *C125.9* ($|S|=34$)

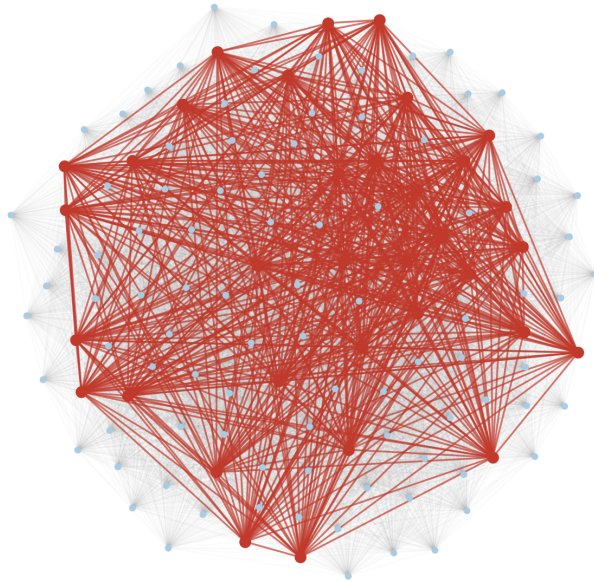


Figura 4: Clique de tamaño 34 encontrado por Tabú+ILS en *C125.9*.

5. Contraste de Resultados: Método Exacto vs. Metaheurística

Esta sección contrasta explícitamente los resultados de las dos soluciones algorítmicas implementadas para el MCP: el modelo ILP resuelto por Ramificación y Acotamiento del Informe 1, y la Búsqueda Tabú con reinicio de este informe.

5.1. Fuentes de comparación

Para *keller4* se usa directamente el resultado reportado en el Informe 1: modelo ILP (Pyomo) resuelto con HiGHS, que certificó $\omega = 11$ en 12–18 s. Para *C125.9* no se reportó un resultado formal en el Informe 1 (que se centró en *keller4*); se utiliza en su lugar un experimento exploratorio realizado durante el desarrollo de dicho informe con un Branch & Bound manual (framework PyBnB (Hackebeil, 2020)) sobre la relajación LP, ejecutado con un límite de 300 s. Este experimento es la referencia exacta más directa disponible sobre una instancia más densa que *keller4*, y se documenta aquí junto con sus resultados para dejar la comparación completamente trazable. El notebook ejecutable con el código y las salidas completas de este experimento (incluyendo los 232 953 nodos explorados y el estado *feasible* final) se encuentra en el archivo `notebook/maximum_clique_bb.ipynb`, incluido junto con la entrega de este trabajo.

Nota metodológica sobre las dos referencias exactas. Ambas referencias exactas no son metodológicamente idénticas entre sí, lo cual es relevante para interpretar el contraste. HiGHS es un solver MIP industrial que, internamente, combina la Ramificación y Acotamiento con planos de corte y heurísticas de factibilidad (Informe 1, Sección 3.2); su tiempo de 12–18 s en *keller4* refleja, por tanto, un B&B *asistido por heurísticas*, no una Ramificación y Acotamiento pura. El B&B manual usado como referencia en *C125.9*, en cambio, sí corresponde a una Ramificación y Acotamiento sin heurísticas ni cortes adicionales: en cada nodo se resuelve únicamente la relajación LP (mediante `scipy`) y se ramifica sobre una variable fraccionaria, sin mecanismos de aceleración adicionales. Esta segunda referencia es, en consecuencia, la comparación más

limpia entre búsqueda exacta pura y metaheurística de las dos presentadas en esta sección. Asimismo, a diferencia del Informe 1 —que evaluó una única instancia—, este informe contrasta ambos enfoques sobre dos instancias de tamaño y densidad distintos (171 vs. 125 vértices; densidad 0.65 vs. 0.90), lo que permite observar cómo se comporta cada método al variar la dificultad de la instancia y no solo su tamaño nominal.

5.2. Tabla comparativa

La Tabla 4 resume ambos enfoques sobre las dos instancias.

Tabla 4: Contraste entre el método exacto (Informe 1) y la metaheurística (Informe 2).

Inst.	Método	ω	Tiempo	¿Óptimo?	Notas
keller4	ILP + HiGHS (Informe 1)	11	12–18 s	Sí	Único resultado reportado
	Tabú + ILS (Informe 2)	11	0.021 s	No	Media 10.90; 28/30 corridas óptimas
C125.9	B&B manual, PyBnB	34	300 s (<i>timeout</i>)	No	232 953 nodos explorados
	Tabú + ILS (Informe 2)	34	0.025 s	No	Media 33.97; 29/30 corridas óptimas

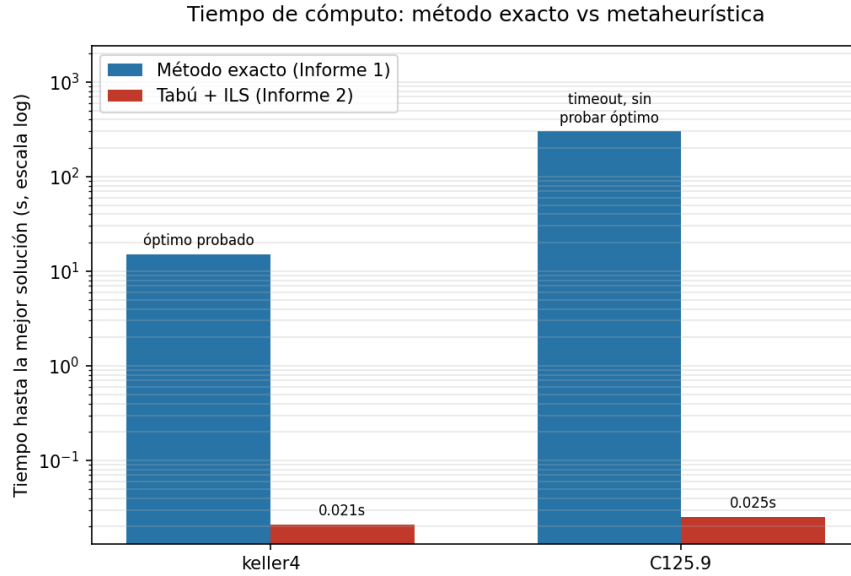


Figura 5: Tiempo hasta la mejor solución: método exacto vs. metaheurística (escala logarítmica).

5.3. Análisis del contraste

Calidad de la solución. En ambas instancias, la mejor solución encontrada por la metaheurística en al menos una corrida coincide exactamente con la del método exacto ($\omega = 11$ y $\omega = 34$, respectivamente). Considerando el conjunto de las 30 corridas, la calidad media se mantiene muy cercana al óptimo (a 0.10 y 0.03 vértices de distancia en promedio, respectivamente), lo que indica que incluso en las corridas que no alcanzan el óptimo, la solución obtenida es prácticamente óptima en términos prácticos.

Tiempo de cómputo. La diferencia de tiempos es de varios órdenes de magnitud. En *keller4*, la metaheurística encuentra su mejor solución, en promedio, cerca de 711 veces más rápido que el tiempo que HiGHS tarda en *certificar* el óptimo (0.021s vs. 15s, tomando el punto medio del rango reportado). En *C125.9* la brecha es aún mayor: el B&B manual agota los 300s sin certificar optimalidad, mientras que la metaheurística encuentra su mejor solución en 0.025s en promedio —más de 11 900 veces más rápido—, aunque, a diferencia del caso anterior, ninguno de los dos métodos certifica formalmente

que 34 sea el óptimo en esta comparación (el valor $\omega = 34$ se toma como óptimo conocido de la instancia según el benchmark DIMACS (Mascia, 2014), no como algo demostrado por ninguno de los dos algoritmos aquí ejecutados).

El compromiso central: garantía vs. escalabilidad. El contraste evidencia el trade-off fundamental entre ambos paradigmas, ya anticipado en la introducción de este informe. El método exacto ofrece una *garantía*: cuando termina con estado *óptimo*, no existe clique más grande en el grafo. Esa garantía tiene un costo computacional que crece de forma no controlada con la dificultad de la instancia —en *keller4* son segundos, pero en *C125.9*, con una densidad considerablemente mayor (0.90 vs. 0.65) y menos vértices, el mismo tipo de método ya no logra concluir en 300 s—. La metaheurística invierte esta relación: no ofrece garantía de optimalidad en ninguna corrida individual, pero su tiempo de cómputo es prácticamente insensible a la dificultad de la instancia (0.021 s y 0.025 s son estadísticamente indistinguibles pese a que *C125.9* es la instancia donde el método exacto colapsa). La garantía perdida se recupera parcialmente mediante repetición: al ejecutar múltiples corridas independientes y quedarse con la mejor, la probabilidad empírica de *no* encontrar el óptimo cae rápidamente (con una tasa de éxito por corrida de 93–97 %, y suponiendo corridas independientes, la probabilidad de que las 30 corridas fallen simultáneamente es inferior a 10^{-30}).

Cuándo preferir cada enfoque. A la luz de estos resultados, el método exacto resulta preferible cuando se requiere una garantía formal de optimalidad y la instancia es lo suficientemente pequeña o estructurada para que el árbol de Branch & Bound sea manejable (como *keller4*). La metaheurística resulta preferible cuando el tiempo de respuesta es crítico, quiere explorarse un gran número de instancias, o la instancia es lo suficientemente grande o densa para que el enfoque exacto sea inviable en la práctica —exactamente el escenario ilustrado por *C125.9*—, aceptando a cambio una solución sin certificado formal de optimalidad, aunque empíricamente de calidad prácticamente óptima.

6. Discusión

6.1. Efectividad de la Búsqueda Tabú con reinicio

Los resultados de la Sección 4 muestran que una metaheurística relativamente simple —construida sobre apenas tres tipos de movimiento y una lista tabú de tamaño fijo— alcanza el óptimo conocido en más del 93 % de las corridas en ambas instancias, con tiempos de cómputo del orden de centésimas de segundo. Esto confirma, para el caso particular del MCP sobre estas dos instancias, observaciones ya reportadas en la literatura (Battiti & Protasi, 2001; Pullan & Hoos, 2006): la combinación de una vecindad que preserva la factibilidad por construcción (Sección 3.1) con un mecanismo simple de diversificación es suficiente para obtener soluciones de calidad prácticamente óptima sin necesidad de estructuras de datos ni esquemas de aceptación más sofisticados (p. ej. recocido simulado con esquemas de enfriamiento, o tabú reactivo con *tenure* adaptativo).

6.2. Análisis de las corridas que no alcanzaron el óptimo

Las tres corridas que no alcanzaron $\omega(G)$ (semillas 5 y 14 en *keller4*, semilla 24 en *C125.9*) comparten un patrón que descarta que la causa haya sido una falta de oportunidades de perturbación: las tres alcanzan su mejor solución muy temprano —en las iteraciones 2, 11 y 31 de un máximo de 8 000, respectivamente— y aun así completan 39 reinicios cada una, un número prácticamente idéntico al rango que exhiben las corridas que sí alcanzan el óptimo (36–40 reinicios, combinando ambas instancias). Es decir, cada una de estas tres corridas ejecutó 39 perturbaciones adicionales después de alcanzar su mejor tamaño, sin conseguir mejorarlo en ninguna de ellas. Esto sugiere que el fallo no se debe a que la fracción de perturbación (10–30 % de S) haya dispuesto de pocos intentos, sino a que, para la trayectoria pseudoaleatoria particular de esas semillas, la reconstrucción voraz posterior a cada perturbación converge reiteradamente hacia la misma región subóptima del espacio de búsqueda —consistente con una cuenca de atracción (*plateau*) lo bastante amplia como para no ser evadida por una perturbación

parcial, incluso tras varias decenas de intentos independientes entre sí.

6.3. Rol del mecanismo de reinicio

El experimento de ablación de la Sección 4 (Tabla 3) aísla el aporte específico de la perturbación tipo ILS: sin ella, la tasa de éxito cae 20–30 puntos porcentuales y la desviación estándar del tamaño final aumenta notoriamente (cerca del doble en *keller4*, y más de seis veces en *C125.9*). Esto es consistente con la motivación teórica de Lourenço et al. (2003): una búsqueda local de trayectoria única, por muy bien diseñada que esté su vecindad, tiene una probabilidad no despreciable de estancarse en una región del espacio de búsqueda; una perturbación periódica que rompe parcialmente la solución incumbente y la reconstruye permite «reiniciar» la búsqueda en una región distinta sin descartar por completo el progreso acumulado. El hecho de que el efecto sea más marcado en *C125.9* (instancia más densa, con más movimientos SWAP disponibles y por tanto más plateaus) sugiere que la utilidad de este mecanismo crece con la dificultad combinatoria de la instancia, justo el régimen donde más se necesita.

6.4. Ventajas del enfoque implementado

- **Independencia de un solver externo.** A diferencia del Informe 1, que dependía de HiGHS o GLPK, la metaheurística es autocontenida: toda su lógica es código Python propio sobre NumPy, sin dependencias de solvers comerciales o de código abierto especializados en programación matemática.
- **Costo por iteración acotado.** Gracias a la vectorización de $\deg_S(v)$ mediante productos matriciales (Sección 3.1), cada iteración cuesta $O(nk)$, independientemente de cuán difícil sea la instancia para un solver MIP; esto explica por qué el tiempo de cómputo de la metaheurística es casi idéntico en *keller4* y *C125.9* pese a la enorme diferencia de dificultad que ambas presentan para el enfoque exacto.
- **Facilidad de paralelización.** Al ser las 30 corridas completamente independientes entre sí (distintas semillas, sin comunicación), el protocolo experimental

es trivialmente paralelizable, algo que no ocurre de forma natural en un único árbol de Branch & Bound.

6.5. Comparación con otros diseños de metaheurística considerados

Durante el diseño se consideraron dos alternativas a la combinación Tabú+ILS finalmente implementada:

1. **Búsqueda Tabú pura (sin reinicio).** Es la opción más directamente alineada con el diseño de Tarea 1, que enfatiza la lista tabú y el criterio de aspiración. Sin embargo, el experimento de ablación de la Sección 4 muestra que, sin un mecanismo de diversificación adicional, su tasa de éxito se degrada notablemente en la instancia más densa, lo que la vuelve menos confiable para el objetivo de este informe.
2. **VNS (Variable Neighborhood Search) (Hansen & Mladenović, 2007)** sobre $N_1 = \mathbf{ADD}$, $N_2 = \mathbf{DROP}$, $N_3 = \mathbf{SWAP}$. Tarea 1 esboza esta alternativa. Se descartó como diseño principal porque, al no incorporar memoria de movimientos recientes (a diferencia de la lista tabú), es más propensa a ciclar entre un pequeño conjunto de cliques maximales vecinos en instancias densas; queda como línea de trabajo futuro (Sección 7) evaluarla comparativamente bajo el mismo protocolo experimental.

6.6. Limitaciones

- El estudio se limita a dos instancias del benchmark DIMACS; ambas de tamaño moderado ($n < 200$). No se evalúa el comportamiento de la metaheurística en instancias de miles de vértices, donde el costo $O(nk)$ por iteración podría volverse un cuello de botella y donde el comportamiento relativo frente al método exacto podría diferir.

- Los parámetros (`tenure`, `stall_limit`, fracción de perturbación) se fijaron empíricamente mediante un barrido manual acotado, no mediante una estrategia de ajuste reactivo como la de Battiti y Tecchioli (1994). Es posible que un esquema de *tenure* adaptativo mejore aún más la tasa de éxito, en particular en instancias más difíciles que las aquí evaluadas.
- La comparación de tiempos de la Sección 5 contrasta implementaciones con grados de madurez distintos: HiGHS es un solver MIP industrial altamente optimizado (C++), mientras que tanto el B&B manual como la metaheurística de este informe son implementaciones académicas en Python puro/NumPy. Parte de la brecha de tiempos observada podría reducirse con una implementación más eficiente de cualquiera de los métodos, aunque es poco probable que cambie la conclusión cualitativa sobre el orden de magnitud de la diferencia.
- Ninguno de los dos métodos exactos usados como referencia certifica formalmente $\omega = 34$ en *C125.9*; ese valor proviene del benchmark DIMACS (Mascia, 2014) y se trata en este informe como óptimo conocido, no demostrado internamente.

7. Conclusiones

Este informe dio continuidad al estudio del Problema del Clique Máximo iniciado en el Informe 1, abordándolo mediante una metaheurística de búsqueda local —Búsqueda Tabú con reinicio tipo ILS— construida sobre la representación y la vecindad Add/Drop/Swap diseñadas en la Tarea 1, y contrastando sistemáticamente sus resultados con los del método exacto. Se establecen las siguientes conclusiones:

1. **El diseño de Tarea 1 resultó directamente implementable.** La representación de soluciones como cliques explícitos, la función objetivo trivial y el operador de vecindad Add/Drop/Swap —especificados sin código en Tarea 1— se tradujeron sin fricciones en un algoritmo funcional y eficiente, validando la utilidad de separar el diseño conceptual de la metaheurística de su instanciación concreta.
2. **Calidad de solución prácticamente óptima.** La Búsqueda Tabú con reinicio alcanzó el óptimo conocido en el 93.3 % de las corridas en *keller4* ($\omega = 11$) y en el 96.7 % en *C125.9* ($\omega = 34$); en las corridas restantes la solución quedó a lo sumo a dos vértices de distancia del óptimo.
3. **El mecanismo de reinicio es indispensable, no accesorio.** El experimento de ablación realizado mostró que desactivar la perturbación tipo ILS reduce la tasa de éxito en 20–30 puntos porcentuales, confirmando que la diversificación explícita —más allá de la lista tabú— es un componente crítico del diseño, especialmente en instancias densas.
4. **Ventaja de tiempo de varios órdenes de magnitud.** La metaheurística encontró su mejor solución en 0.021–0.025 s en promedio, frente a 12–18 s del método exacto en *keller4* y a un *timeout* de 300 s sin certificar optimalidad en *C125.9*. Esta ventaja se explica por el costo por iteración $O(nk)$, prácticamente insensible a la dificultad combinatoria que sí afecta drásticamente al árbol de Branch & Bound. Esta velocidad es particularmente relevante para los contextos de aplicación identificados en la Sección 2: en tareas como el cribado (*screening*) de subestructuras moleculares comunes para descubrimiento de fármacos, donde se

evalúan muchos pares de moléculas candidatas, un tiempo de respuesta de milisegundos por instancia —en lugar de segundos o minutos— sugiere que este enfoque podría explorar órdenes de magnitud más candidatos que un método exacto bajo el mismo presupuesto de tiempo; confirmar esto a la escala real de esos dominios (miles de vértices, no evaluada en este trabajo) queda como línea de trabajo futuro.

5. **El contraste confirma el compromiso teórico garantía-escalabilidad.** El método exacto certifica la optimalidad a costa de un tiempo de cómputo que se degrada exponencialmente con la dificultad de la instancia; la metaheurística sacrifica esa certeza individual mensurable, pero la compensa con velocidad y con una alta tasa de éxito reproducible mediante corridas independientes. Ninguno de los dos enfoques domina al otro de forma absoluta: la elección depende de si el requisito es una garantía formal o una respuesta rápida de calidad verificable empíricamente.
6. **Reproducibilidad.** La implementación completa (algoritmo, experimentos, generación de figuras) está disponible en `notebook_metaheuristica/` como código Python con dependencias abiertas (NumPy (Harris et al., 2020), NetworkX (Hagberg et al., 2008), Matplotlib (Hunter, 2007)), sin requerir licencias de solvers comerciales.

7.1. Trabajo futuro

- Evaluar el algoritmo sobre instancias DIMACS de mayor tamaño ($n > 500$) para caracterizar el punto en que el costo $O(nk)$ por iteración deja de ser despreciable.
- Comparar Tabú+ILS contra un VNS (Hansen & Mladenović, 2007) sobre las mismas tres vecindades ($N_1 = \text{ADD}$, $N_2 = \text{DROP}$, $N_3 = \text{SWAP}$), esbozado pero no implementado en este trabajo, bajo el mismo protocolo experimental de 30 corridas.

- Incorporar ajuste reactivo del *tenure* (Battiti & Tecchiolli, 1994) en lugar del valor fijo utilizado, y evaluar su efecto en instancias más difíciles que las estudiadas aquí.
- Ejecutar un Branch & Bound formal (HiGHS) sobre *C125.9* —no solo el B&B manual usado como referencia— para obtener una certificación de optimalidad directamente comparable a la de *keller4*.

8. Apéndice

8.1. Entorno de ejecución

Tabla 5: Herramientas de software utilizadas.

Herramienta	Versión	Propósito
Python (Python Software Foundation, 2024)	3.9+	Lenguaje de programación
NumPy (Harris et al., 2020)	2.0+	Cómputo vectorizado sobre la matriz de adyacencia
NetworkX (Hagberg et al., 2008)	3.2+	Visualización de grafos
Matplotlib (Hunter, 2007)	3.9+	Generación de figuras

8.2. Cálculo de la vecindad y construcción voraz

Implementación vectorizada de $C(S)$ y de los pares SWAP (Sección 3.1), y de la construcción voraz inicial:

```
def _candidates(A: np.ndarray, S: np.ndarray):
    """Retorna (add_candidates, swap_pairs) para la clique S.

    add_candidates: array de vértices v fuera de S adyacentes a todo S.
    swap_pairs: lista de (v, u) con v fuera de S, u en S, tal que
                 intercambiar u por v mantiene S como clique.
    """
    k = int(S.sum())
    if k == 0:
        outside = np.where(~S)[0]
        return outside, []
    deg_S = A[:, S].sum(axis=1)          # deg_S[v] = |N(v) intersect S|, para todo v
    outside_mask = ~S
    add_candidates = np.where(outside_mask & (deg_S == k))[0]
    swap_v = np.where(outside_mask & (deg_S == k - 1))[0]
    swap_pairs = []
    S_idx = np.where(S)[0]
    for v in swap_v:
```



```

        # único u en S no adyacente a v
        non_adj = S_idx[~A[v, S_idx]]
        if len(non_adj) == 1:
            swap_pairs.append((int(v), int(non_adj[0])))
    return add_candidates, swap_pairs

def _greedy_construct(A: np.ndarray, n: int, rng: random.Random, seed_S: np.ndarray | None = None):
    """Construye una clique maximal agregando vértices al azar desde C(S)
    hasta que C(S) quede vacío."""
    S = np.zeros(n, dtype=bool) if seed_S is None else seed_S.copy()
    while True:
        add_candidates, _ = _candidates(A, S)
        if len(add_candidates) == 0:
            break
        v = rng.choice(list(add_candidates))
        S[v] = True
    return S

```

8.3. Búsqueda Tabú con reinicio (Algoritmo 1)

La función retorna un objeto `TabuILSResult` (un `dataclass` con los campos `best_S`, `best_size`, `history`, `n_restarts`, `n_iterations` y `elapsed`) en lugar de una tupla simple, de modo que `run_experiments.py` pueda registrar la trayectoria completa de cada corrida (usada en las Figuras 1 y 2) sin variables globales. Los valores por defecto de los parámetros (`max_iterations=20000`, `tenure=12`, `stall_limit=300`) no son los efectivamente usados en los experimentos reportados; `run_experiments.py` siempre los sobrescribe explícitamente con `max_iterations=8 000`, `tenure=10` y `stall_limit=200` (Sección 3).

```

def tabu_ils(
    A: np.ndarray,
    n: int,
    max_iterations: int = 20000,
    tenure: int = 12,

```

```

    stall_limit: int = 300,
    perturb_frac: tuple = (0.1, 0.3),
    time_limit: float | None = None,
    seed: int | None = None,
) -> TabuILSResult:
    """Búsqueda Tabú con reinicio por perturbación (ILS-like) para el MCP.

    Parámetros
    -----
    tenure: número de iteraciones que un vértice recién eliminado
            permanece tabú para re-agregarse.
    stall_limit: iteraciones sin mejorar el mejor incumbente antes de
            aplicar una perturbación (diversificación).
    perturb_frac: fracción (min, max) de vértices de la clique actual
            que se eliminan al perturbar.
    """
    rng = random.Random(seed)
    t0 = time.time()

    S = _greedy_construct(A, n, rng)
    best_S = S.copy()
    best_size = int(S.sum())

    tabu_until = np.zeros(n, dtype=int) # iteración hasta la cual v es tabú (para Add)
    stall = 0
    n_restarts = 0
    history = []

    it = 0
    while it < max_iterations:
        if time_limit is not None and (time.time() - t0) > time_limit:
            break
        it += 1

        add_candidates, swap_pairs = _candidates(A, S)

```

```

if len(add_candidates) > 0:
    # Movimiento siempre mejorante: agregar (respeta tabú salvo aspiración)
    non_tabu = [v for v in add_candidates if tabu_until[v] <= it]
    pool = non_tabu if non_tabu else list(add_candidates) # aspiración: mejora incumbente
    v = rng.choice(pool)
    S[v] = True
    stall = 0
elif swap_pairs:
    # Movimiento lateral: swap que maximiza  $|C(S')|$  resultante (más prometedor)
    non_tabu_pairs = [(v, u) for v, u in swap_pairs if tabu_until[v] <= it]
    pool = non_tabu_pairs if non_tabu_pairs else swap_pairs
    best_pair, best_gain = None, -1
    sample = pool if len(pool) <= 15 else rng.sample(pool, 15)
    for v, u in sample:
        S_try = S.copy()
        S_try[u] = False
        S_try[v] = True
        add_after, _ = _candidates(A, S_try)
        gain = len(add_after)
        if gain > best_gain:
            best_gain, best_pair = gain, (v, u)
    v, u = best_pair
    S[u] = False
    S[v] = True
    tabu_until[u] = it + tenure
    stall += 1
else:
    # Clique maximal sin swaps disponibles: drop forzado (diversifica)
    S_idx = np.where(S)[0]
    if len(S_idx) == 0:
        S = _greedy_construct(A, n, rng)
        continue
    u = rng.choice(list(S_idx))
    S[u] = False
    tabu_until[u] = it + tenure
    stall += 1

```

```

cur_size = int(S.sum())
if cur_size > best_size:
    best_size = cur_size
    best_S = S.copy()
    stall = 0

history.append((it, time.time() - t0, cur_size, best_size))

if stall >= stall_limit:
    # Perturbación (reinicio ILS): eliminar una fracción aleatoria
    # de la clique actual y reconstruir de forma golosa.
    n_restarts += 1
    S_idx = np.where(S)[0]
    frac = rng.uniform(*perturb_frac)
    n_remove = max(1, int(len(S_idx) * frac))
    to_remove = rng.sample(list(S_idx), min(n_remove, len(S_idx)))
    for u in to_remove:
        S[u] = False
        tabu_until[u] = it + tenure
    S = _greedy_construct(A, n, rng, seed_S=S)
    stall = 0

elapsed = time.time() - t0
return TabuILSResult(
    best_S=best_S,
    best_size=best_size,
    history=history,
    n_restarts=n_restarts,
    n_iterations=it,
    elapsed=elapsed,
)

```

8.4. Instancias: formato DIMACS

Ambas instancias se distribuyen en formato ASCII estándar DIMACS, idéntico al utilizado en el Informe 1:

```
c C125.9.clq
p col 125 6963
e 1 2
e 1 5
...
```

Donde `p col 125 6963` define un grafo de 125 vértices y 6963 aristas, y cada línea `e u v` define una arista entre los vértices u y v . Los archivos completos (`keller4.clq`, `C125.9.clq`) y el código completo de los experimentos se encuentran en la carpeta `notebook_metaheuristica/`, incluida junto con la entrega de este trabajo: `mcp_tabu_ils.py` (algoritmo), `run_experiments.py` (30 corridas por instancia, selección determinística de la corrida representativa para las Figuras 1–4) y `run_ablation.py` (experimento de ablación de la Tabla 3). El experimento exacto de *C125.9* referenciado en la Sección 5 está en `notebook/maximum_clique_bb.ipynb`.

Referencias bibliográficas

- Battiti, R., & Protasi, M. (2001). Reactive local search for the maximum clique problem. *Algorithmica*, 29(4), 610-637. <https://doi.org/10.1007/s004530010074>.
- Battiti, R., & Tecchiolli, G. (1994). The reactive tabu search. *ORSA Journal on Computing*, 6(2), 126-140. <https://doi.org/10.1287/ijoc.6.2.126>.
- Bomze, I. M., Budinich, M., Pardalos, P. M., & Pelillo, M. (1999). The maximum clique problem. En D.-Z. Du & P. M. Pardalos (Eds.), *Handbook of Combinatorial Optimization (Supplement Volume A)* (pp. 1-74, Vol. 4). Kluwer Academic Publishers.
- Gardiner, E. J., Artymiuk, P. J., & Willett, P. (1997). Clique-detection algorithms for matching three-dimensional molecular structures. *Journal of Molecular Graphics and Modelling*, 15(4), 245-253. [https://doi.org/10.1016/S1093-3263\(97\)00089-2](https://doi.org/10.1016/S1093-3263(97)00089-2).
- Glover, F. (1989). Tabu search—part I. *ORSA Journal on Computing*, 1(3), 190-206. <https://doi.org/10.1287/ijoc.1.3.190>.
- Glover, F. (1990). Tabu search—part II. *ORSA Journal on Computing*, 2(1), 4-32. <https://doi.org/10.1287/ijoc.2.1.4>.
- Glover, F., & Laguna, M. (1997). *Tabu Search*. Kluwer Academic Publishers, Boston.
- Hackebeil, G. A. (2020). PyBnB: A parallel branch-and-bound engine for Python [MIT License].
- Hagberg, A. A., Schult, D. A., & Swart, P. J. (2008). Exploring network structure, dynamics, and function using NetworkX. *Proceedings of the 7th Python in Science Conference (SciPy2008)*, 11-15.
- Hansen, P., & Mladenović, N. (2007). *Variable Neighborhood Search Methods* (inf. téc. N.º G-2007-52). Les Cahiers du GERAD.
- Harris, C. R., Millman, K. J., van der Walt, S. J., et al. (2020). Array programming with NumPy. *Nature*, 585, 357-362. <https://doi.org/10.1038/s41586-020-2649-2>.
- Huangfu, Q., & Hall, J. A. J. (2018). Parallelizing the dual revised simplex method. *Mathematical Programming Computation*, 10(1), 119-142. <https://doi.org/10.1007/s12532-017-0130-5>.

- Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*, 9(3), 90-95. <https://doi.org/10.1109/MCSE.2007.55>.
- Johnson, D. S., & Trick, M. A. (1993). The Second DIMACS Implementation Challenge: NP Hard Problems: Maximum Clique, Graph Coloring, and Satisfiability.
- Katayama, K., Hamamoto, A., & Narisawa, H. (2005). An effective local search for the maximum clique problem. *Information Processing Letters*, 95(5), 503-511. <https://doi.org/10.1016/j.ipl.2005.05.010>.
- Lagarias, J. C., & Shor, P. W. (1992). Keller's cube-tiling conjecture is false in high dimensions. *Bulletin of the American Mathematical Society*, 27(2), 279-283. <https://doi.org/10.1090/S0273-0979-1992-00318-X>.
- Lourenço, H. R., Martin, O. C., & Stützle, T. (2003). Iterated Local Search. En F. Glover & G. A. Kochenberger (Eds.), *Handbook of Metaheuristics* (pp. 321-353, Vol. 57). Kluwer Academic Publishers. https://doi.org/10.1007/0-306-48056-5_11.
- Mascia, F. (2014). DIMACS Benchmark Set for the Maximum Clique Problem [IRIDIA, Université Libre de Bruxelles. Consultado en abril 2026].
- Pardalos, P. M., & Xue, J. (1994). The maximum clique problem. *Journal of Global Optimization*, 4(3), 301-328. <https://doi.org/10.1007/bf01098364>.
- Pullan, W., & Hoos, H. H. (2006). Dynamic local search for the maximum clique problem. *Journal of Artificial Intelligence Research*, 25, 159-185. <https://doi.org/10.1613/jair.1815>.
- Python Software Foundation. (2024). Python Language Reference, version 3.
- Singh, K. K., & Govinda, L. (2014). A Simple and Efficient Heuristic Algorithm For Maximum Clique Problem. *2014 IEEE 8th International Conference on Intelligent Systems and Control (ISCO)*, 269-273. <https://doi.org/10.1109/ISCO.2014.7103958>.